

Introduction to Haskell Programming

Built-in functions on lists, list notations

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Built-in functions on lists

head (x:xs) = x

tail (x:xs) = xs

length [] = 0

length (x:xs) = 1 + **length** xs

sum [] = 0

sum (x:xs) = x + **sum** xs

Built-in functions on lists

- `init` returns all but the last element of a list

Built-in functions on lists

- `init` returns all but the last element of a list
- `last` returns the last element of a list

Built-in functions on lists

- `init` returns all but the last element of a list
- `last` returns the last element of a list
- Undefined for the empty list

Built-in functions on lists

- **init** returns all but the last element of a list
- **last** returns the last element of a list
- Undefined for the empty list
- Possible implementations:

```
init [x]      = []  
init (x:xs)   = x:init xs  
  
last [x]      = x  
last (x:xs)   = last xs
```

Built-in functions on lists

head (x:xs) = x

tail (x:xs) = xs

length [] = 0

length (x:xs) = 1 + **length** xs

sum [] = 0

sum (x:xs) = x + **sum** xs

Built-in functions on lists

- `init` returns all but the last element of a list

Built-in functions on lists

- `init` returns all but the last element of a list
- `last` returns the last element of a list

Built-in functions on lists

- `init` returns all but the last element of a list
- `last` returns the last element of a list
- Undefined for the empty list

Built-in functions on lists

- **init** returns all but the last element of a list
- **last** returns the last element of a list
- Undefined for the empty list
- Possible implementations:

```
init [x]      = []  
init (x:xs)   = x:init xs
```

```
last [x]      = x  
last (x:xs)   = last xs
```

Built-in functions on lists

- `take n l` returns the first `n` elements of `l`

Built-in functions on lists

- **take** *n* *l* returns the first *n* elements of *l*
- **drop** *n* *l* returns all but the first *n* elements of *l*

Built-in functions on lists

- **take** *n* *l* returns the first *n* elements of *l*
- **drop** *n* *l* returns all but the first *n* elements of *l*
- **take** *n* *l* ++ **drop** *n* *l* = *l*

```
take _ [] = []  
take n (x:xs) | n ≤ 0 = []  
              | otherwise = x:take (n-1) xs
```

```
drop _ [] = []  
drop n (x:xs) | n ≤ 0 = x:xs  
              | otherwise = drop (n-1) xs
```

Built-in function: `splitAt`

- First attempt:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
    ([], x:xs) else
    (x:fst (splitAt (n-1) xs), snd (splitAt (n-1) xs))
```

Built-in function: `splitAt`

- First attempt:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
    ([], x:xs) else
    (x:fst (splitAt (n-1) xs), snd (splitAt (n-1) xs))
```

- Two recursive calls to `splitAt (n-1)`

Built-in function: `splitAt`

- First attempt:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
    ([], x:xs) else
    (x:fst (splitAt (n-1) xs), snd (splitAt (n-1) xs))
```

- Two recursive calls to `splitAt (n-1)`
- Wasteful repetition of computation!

Built-in function: `splitAt`

- First attempt:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n <= 0 then
    ([], x:xs) else
    (x:fst (splitAt (n-1) xs), snd (splitAt (n-1) xs))
```

- Two recursive calls to `splitAt (n-1)`
- Wasteful repetition of computation!
- Actually `ghci` does some optimizations to make only one recursive call, **but we cannot rely on this happening all the time!**

Built-in function: `splitAt`

- Much better version:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
  ([], x:xs) else
  let (ys, zs)    = splitAt (n-1) xs
  in (x:ys, zs)
```

Built-in function: `splitAt`

- Much better version:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
    ([], x:xs) else
    let (ys, zs)   = splitAt (n-1) xs
    in (x:ys, zs)
```

- Only one recursive call to `splitAt (n-1)`

Built-in function: `splitAt`

- Much better version:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
  ([], x:xs) else
  let (ys, zs)    = splitAt (n-1) xs
  in (x:ys, zs)
```

- Only one recursive call to `splitAt (n-1)`
- Running time is proportional to n

Built-in function: `splitAt`

- Much better version:

```
splitAt _ []      = ([], [])
splitAt n (x:xs)  = if n ≤ 0 then
    ([], x:xs) else
    let (ys, zs)   = splitAt (n-1) xs
    in (x:ys, zs)
```

- Only one recursive call to `splitAt (n-1)`
- Running time is proportional to n
- **Local definitions helps avoid repeated computation of same value**

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`
- Positions in any list are numbered from `0` to `length l - 1`

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`
- Positions in any list are numbered from `0` to `length l - 1`
- `l !! j` is the value at position `j` of the list

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`
- Positions in any list are numbered from `0` to `length l - 1`
- `l !! j` is the value at position `j` of the list
- Accessing position `j` takes time proportional to `j`

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`
- Positions in any list are numbered from `0` to `length l - 1`
- `l !! j` is the value at position `j` of the list
- Accessing position `j` takes time proportional to `j`
- Need to “peel off” applications of the `:` operator

List notation

- `valueAtPosition` is equivalent to the built-in operator `!!`
- Positions in any list are numbered from `0` to `length l - 1`
- `l !! j` is the value at position `j` of the list
- Accessing position `j` takes time proportional to `j`
- Need to “peel off” applications of the `:` operator
- Arrays, in other languages, allow constant-time access to any position

List notation

- $[m .. n] \text{ ---> } [m, m+1, \dots, n]$

List notation

- $[m .. n] \rightarrow [m, m+1, \dots, n]$
- Returns empty list if $m > n$

List notation

- $[m .. n] \rightarrow [m, m+1, \dots, n]$
- Returns empty list if $m > n$
 - $[1 .. 7] \rightarrow [1, 2, 3, 4, 5, 6, 7]$

List notation

- $[m .. n] \text{ ---> } [m, m+1, \dots, n]$
- Returns empty list if $m > n$
 - $[1 .. 7] \text{ ---> } [1, 2, 3, 4, 5, 6, 7]$
 - $[3 .. 3] \text{ ---> } [3]$

List notation

- $[m .. n] \text{ ---> } [m, m+1, \dots, n]$
- Returns empty list if $m > n$
 - $[1 .. 7] \text{ ---> } [1, 2, 3, 4, 5, 6, 7]$
 - $[3 .. 3] \text{ ---> } [3]$
 - $[5 .. 4] \text{ ---> } []$

List notation

- $[m \dots n] \rightarrow [m, m+1, \dots, n]$
- Returns empty list if $m > n$
 - $[1 \dots 7] \rightarrow [1, 2, 3, 4, 5, 6, 7]$
 - $[3 \dots 3] \rightarrow [3]$
 - $[5 \dots 4] \rightarrow []$
- Can skip values (arithmetic progression)

List notation

- `[m .. n] ---> [m, m+1, ... , n]`
- Returns empty list if `m > n`
 - `[1 .. 7] ---> [1,2,3,4,5,6,7]`
 - `[3 .. 3] ---> [3]`
 - `[5 .. 4] ---> []`
- Can skip values (arithmetic progression)
 - `[1,3 .. 8] ---> [1,3,5,7]`

List notation

- `[m .. n] ---> [m, m+1, ... , n]`
- Returns empty list if `m > n`
 - `[1 .. 7] ---> [1,2,3,4,5,6,7]`
 - `[3 .. 3] ---> [3]`
 - `[5 .. 4] ---> []`
- Can skip values (arithmetic progression)
 - `[1,3 .. 8] ---> [1,3,5,7]`
 - `[2,5 .. 19] ---> [2,5,8,11,14,17]`

List notation

- `[m .. n] ---> [m, m+1, ... , n]`
- Returns empty list if `m > n`
 - `[1 .. 7] ---> [1,2,3,4,5,6,7]`
 - `[3 .. 3] ---> [3]`
 - `[5 .. 4] ---> []`
- Can skip values (arithmetic progression)
 - `[1,3 .. 8] ---> [1,3,5,7]`
 - `[2,5 .. 19] ---> [2,5,8,11,14,17]`
- Can have descending sequences

List notation

- `[m .. n] ---> [m, m+1, ... , n]`
- Returns empty list if `m > n`
 - `[1 .. 7] ---> [1,2,3,4,5,6,7]`
 - `[3 .. 3] ---> [3]`
 - `[5 .. 4] ---> []`
- Can skip values (arithmetic progression)
 - `[1,3 .. 8] ---> [1,3,5,7]`
 - `[2,5 .. 19] ---> [2,5,8,11,14,17]`
- Can have descending sequences
 - `[8,7 .. 5] ---> [8,7,6,5]`

List notation

- `[m .. n] ---> [m, m+1, ... , n]`
- Returns empty list if `m > n`
 - `[1 .. 7] ---> [1,2,3,4,5,6,7]`
 - `[3 .. 3] ---> [3]`
 - `[5 .. 4] ---> []`
- Can skip values (arithmetic progression)
 - `[1,3 .. 8] ---> [1,3,5,7]`
 - `[2,5 .. 19] ---> [2,5,8,11,14,17]`
- Can have descending sequences
 - `[8,7 .. 5] ---> [8,7,6,5]`
 - `[12,8 .. (-9)] ---> [12,8,4,0,-4,-8]`

New lists from old

- Set comprehension in mathematics

New lists from old

- Set comprehension in mathematics
- $M = \{x^2 \mid x \in L, \text{even}(x)\}$

New lists from old

- Set comprehension in mathematics
- $M = \{x^2 \mid x \in L, \text{even}(x)\}$
- Generates a new set M from a given set L

New lists from old

- Set comprehension in mathematics
- $M = \{x^2 \mid x \in L, \text{even}(x)\}$
- Generates a new set M from a given set L
- Haskell allows this almost verbatim:

```
m = [x^2 | x <- l, even x]
```

Examples

- All divisors of x

```
divisors x = [y | y <- [1..x], x `mod` y == 0]
```

Examples

- All divisors of x

```
divisors x = [y | y <- [1..x], x `mod` y == 0]
```

- All primes below x

```
primes x = [y | y <- [1..x], divisors y == [1,y]]
```

Examples

- Can use multiple generators

Examples

- Can use multiple generators
- Pairs of integers below 10

```
[(x,y) | x <- [1..10], y <- [1..10]]
```

Examples

- Can use multiple generators
- Pairs of integers below 10

```
[(x,y) | x <- [1..10], y <- [1..10]]
```

- Like nested loops, later generators move faster

```
[(1,1), (1,2), ... , (1,10), (2,1), ... , (2,10),  
... , (10,1), ... , (10,10)]
```

Examples

- All Pythagorean triples below 100

```
[(x,y,z) | x <- [1..100],  
           y <- [1..100],  
           z <- [1..100],  
           x^2 + y^2 == z^2]
```

Examples

- All Pythagorean triples below 100

```
[(x,y,z) | x <- [1..100],  
           y <- [1..100],  
           z <- [1..100],  
           x^2 + y^2 == z^2]
```

- Oops, that has duplicates!

```
[(x,y,z) | x <- [1..100],  
           y <- [(x+1)..100],  
           z <- [(y+1)..100],  
           x^2 + y^2 == z^2]
```

Examples

- All Pythagorean triples below 100

```
[(x,y,z) | x <- [1..100],  
           y <- [1..100],  
           z <- [1..100],  
           x^2 + y^2 == z^2]
```

- Oops, that has duplicates!

```
[(x,y,z) | x <- [1..100],  
           y <- [(x+1)..100],  
           z <- [(y+1)..100],  
           x^2 + y^2 == z^2]
```

- Later lists can refer to earlier generators

Examples

- The built-in function **concat**

```
concat ls = [x | l ← ls, x ← l]
```

Examples

- The built-in function **concat**

```
concat ls = [x | l ← ls, x ← l]
```

- Given a list of lists, extract the head of all even-length non-empty lists

```
headEvens ls = [head l | l ← ls, length l > 0,  
                        even (length l)]
```

Examples

- The built-in function **concat**

```
concat ls = [x | l ← ls, x ← l]
```

- Given a list of lists, extract the head of all even-length non-empty lists

```
headEvens ls = [head l | l ← ls, length l > 0,  
                        even (length l)]
```

- Can use patterns instead of names

```
headEvens ls = [x | (x:xs) ← ls,  
                  even (length (x:xs))]
```